EWB CLOUD LABS • STUDENT PROJECT WORKBOOK

AWS VPC Peering Project

Connect Two VPCs & Prove Private Cross-Network Connectivity

Candidate Name:	Ankit Singh Yadav
Project Title:	AWS VPC Peering — Private Workloads Connectivity
Certification Track:	AWS Certified Solutions Architect Associate
AWS Region:	ap-south-1 (Mumbai)
Completion Date:	October 2026
Evaluator / Program:	EWB Cloud Labs / AWS Internship Assessment

Executive Summary & Evaluation Objectives:

This project workbook demonstrates a secure, multi-VPC networking pattern in Amazon Web Services. Two non-overlapping VPCs (VPC A: 10.10.0.0/16 and VPC B: 10.20.0.0/16) are connected via an active AWS VPC Peering connection. Workloads in private subnets communicate exclusively over private IPv4 addresses without traversing the public internet. Administrative access is established through a hardened bastion host in VPC A using SSH ProxyJump. All requirements, test proofs, negative controls, and viva architectural explanations are documented herein.

1. Architecture & Resource Inventory

1.1 High-Level Architecture

• **VPC A (ewb-vpc-a — 10.10.0.0/16):** Houses public admin subnet (ewb-a-public: 10.10.1.0/24) with ewb-bastion (10.10.1.10 + Public IP) and attached Internet Gateway (ewb-igw-a), plus private workload subnet (ewb-a-private: 10.10.2.0/24) hosting ewb-app-a (10.10.2.10).
• **VPC B (ewb-vpc-b — 10.20.0.0/16):** Strictly private VPC with private subnet (ewb-b-private: 10.20.1.0/24) hosting ewb-app-b (10.20.1.10). VPC B has NO internet gateway.
• **VPC Peering (ewb-a-b-peer):** Direct AWS backbone interconnect bridging VPC A and VPC B for private bidirectional traffic.

1.2 Resource Inventory Table

Resource Name	Type	Resource ID	IPv4 CIDR / Address	Security / Route Note
ewb-vpc-a	VPC	`vpc-0a1b2c3d4e5f6001a`	10.10.0.0/16	Requester VPC; holds IGW & Bastion
ewb-vpc-b	VPC	`vpc-0b2c3d4e5f6a7002b`	10.20.0.0/16	Accepter VPC; completely private
ewb-a-public	Subnet	`subnet-0111aaaa1111bbbb1`	10.10.1.0/24	Public; 0.0.0.0/0 -> IGW
ewb-a-private	Subnet	`subnet-0222aaaa2222bbbb2`	10.10.2.0/24	Private; 10.20.0.0/16 -> PCX
ewb-b-private	Subnet	`subnet-0333aaaa3333bbbb3`	10.20.1.0/24	Private; 10.10.0.0/16 -> PCX (Return)
ewb-bastion	EC2	`i-0123456789abcdef0`	10.10.1.10 (Public IP)	SSH :22 from Admin /32 only
ewb-app-a	EC2	`i-0123456789abcdef1`	10.10.2.10 (No Public IP)	SSH from Bastion; ICMP from App-B
ewb-app-b	EC2	`i-0123456789abcdef2`	10.20.1.10 (No Public IP)	SSH from Bastion; ICMP from App-A
ewb-a-b-peer	VPC Peering	`pcx-0123456789abcdef3`	VPC A <-> VPC B	Active peering connection
ewb-igw-a	Internet GW	`igw-0123456789abcdef4`	Attached to VPC A	Default egress for public subnet

2. Route Tables & Security Group Engineering

2.1 Custom Route Tables Configuration

Route Table	Associated Subnet	Destination CIDR	Target	Architectural Purpose

ewb-rt-a-public	ewb-a-public (10.10.1.0/24)	`10.10.0.0/16`
		`0.0.0.0/0`
		`10.20.0.0/16`
			`local`
			`ewb-igw-a`
			`pcx-...`
				Local VPC routing
				Bastion internet access
				ProxyJump reachability to VPC B

ewb-rt-a-private	ewb-a-private (10.10.2.0/24)	`10.10.0.0/16` `10.20.0.0/16`	`local` `pcx-...`	Local VPC routing Forward path to VPC B workloads
ewb-rt-b-private	ewb-b-private (10.20.1.0/24)	`10.20.0.0/16` `10.10.0.0/16`	`local` `pcx-...`	Local VPC routing **Return path** for responses to VPC A

2.2 Security Group Least-Privilege Rules

Security Group	Inbound Rule Type	Port / Protocol	Source (/32 Host Specific)	Security Justification
ewb-sg-bastion	SSH	TCP 22	`Admin_IP/32`	Restricted exclusively to administrator's public IP
ewb-sg-app-a	SSH Custom ICMP	TCP 22 Echo Request (8)	`10.10.1.10/32` `10.20.1.10/32`	SSH allowed only from Bastion private IP. Ping queries allowed only from App-B.
ewb-sg-app-b	SSH Custom ICMP	TCP 22 Echo Request (8)	`10.10.1.10/32` `10.10.2.10/32`	SSH via ProxyJump from Bastion private IP. Ping queries allowed only from App-A.

3. Verification Testing & Terminal Execution Proofs

Test 01: Baseline Failure (Pre-Peering Routing Isolation)

Objective: Prove that creating a peering connection does not automatically grant connectivity without explicit route tables.

Terminal Execution inside ewb-app-a (10.10.2.10):

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3003ms
```

Result: ACCEPTED. Baseline confirmed dropped packets before routes were associated.

Test 02: Forward ICMP Path (App-A -> App-B via Peering)

Objective: Validate private forward routing across the active peering link with 0% packet loss.

Terminal Execution inside ewb-app-a (10.10.2.10):

```
[ec2-user@ewb-app-a ~]$ hostname && hostname -I
ewb-app-a
10.10.2.10
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=255 time=0.482 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=255 time=0.415 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=255 time=0.428 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=255 time=0.431 ms
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 0.415/0.439/0.482/0.026 ms
```

Result: ACCEPTED. Forward route and return path fully operational.

Test 03: Reverse ICMP Path (App-B -> App-A Bidirectional Proof)

Objective: Prove full duplex cross-VPC communication initiating from VPC B back into VPC A.

Terminal Execution inside ewb-app-b (10.20.1.10):

```
[ec2-user@ewb-app-b ~]$ hostname && hostname -I
ewb-app-b
10.20.1.10
[ec2-user@ewb-app-b ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=255 time=0.450 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=255 time=0.420 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=255 time=0.418 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=255 time=0.425 ms
--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
```

Result: ACCEPTED. Two-way peering reachability verified.

Test 04: Secure Administration via SSH ProxyJump

Objective: Admin connects directly to VPC B's private instance through the bastion without exposing private keys or private IPs to the internet.

Terminal Execution from Local Laptop:

```
$ ssh -i ~/.ssh/ewb-lab.pem -J ec2-user@13.233.x.x ec2-user@10.20.1.10
Last login: Fri Oct 30 16:30:00 2026 from 10.10.1.10
```

```
[ec2-user@ewb-app-b ~]$ hostname -I
10.20.1.10
```

Result: ACCEPTED. ProxyJump utilizes bastion transit hop with source IP 10.10.1.10.

4. Controlled Failure Experiment & Security Analysis

4.1 Experiment Methodology

To isolate and prove the distinct roles of AWS Route Tables versus Security Groups, a negative control test was executed. The peering connection and all route tables remained untouched. Only the inbound ICMP Echo Request rule on ewb-sg-app-b was removed.

4.2 Step 1: Inbound ICMP Rule Deletion (Negative Control)

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3003ms
```

Technical Cause: Although the VPC routing layer correctly forwards packets to VPC B, App-B's elastic network interface (ENI) enforces stateful security group inspection. With no rule matching ICMP Type 8, the packet was dropped silently at the virtual firewall layer.

4.3 Step 2: Inbound ICMP Rule Restoration (Recovery Verification)

```
[ec2-user@ewb-app-a ~]$ ping -c 4 10.20.1.10
64 bytes from 10.20.1.10: icmp_seq=1 ttl=255 time=0.420 ms
4 packets transmitted, 4 received, 0% packet loss
```

Conclusion: Network connectivity requires both a valid route table path (network plumbing) AND explicit security group allowances (host firewall). Security group rule updates take effect immediately in AWS without instance reboot or connection resets.

5. Viva Voce Defense & Architectural Answers

Q1: Why is the return route required in VPC B's route table?
Answer: AWS VPC routing is strictly directional and explicit. While a peering connection establishes the physical/logical conduit between VPCs, packets are routed independently in each direction. When App-A transmits an ICMP query to App-B, VPC A's route table points 10.20.0.0/16 to the peering ID. App-B receives the packet and formulates an Echo Reply directed to 10.10.2.10. Without a return route in ewb-rt-b-private pointing 10.10.0.0/16 to the peering ID, VPC B's router has no destination for the reply and drops it immediately. Hence, return routing is mandatory for bidirectional IP transport.

Q2: Why is the SSH source for App-B set to the bastion private IP, not public IP?
Answer: When traffic originates from an EC2 instance in a public subnet directed toward an internal VPC CIDR or peered VPC, the instance always sends packets using its primary private IPv4 address (10.10.1.10). The Internet Gateway performs 1:1 NAT only for packets destined for public internet endpoints (0.0.0.0/0). Because the peering connection routes internally, packets arriving at App-B carry a source IP of 10.10.1.10. If the security group permitted only the bastion's public IP, App-B's firewall would reject the connection.

Q3: Why does an Active peering connection not guarantee a successful ping?
Answer: 'Active' status merely signifies that the peering handshake has been accepted by both VPCs. Connectivity depends on four distinct layers: (1) Route tables in both VPCs directing target CIDRs to the peering ID; (2) Security Groups on the receiving instance permitting inbound ICMP Type 8; (3) Subnet Network ACLs permitting inbound and outbound ICMP; and (4) The target OS having an active network stack and responding to ICMP. Peering is the link; routing and firewall policies govern transit.

Q4: Can VPC B use VPC A's Internet Gateway over the peering connection?
Answer: No. AWS VPC Peering is non-transitive and expressly prohibits edge routing. Traffic cannot enter a VPC via a peering connection and exit through an attached Internet Gateway, NAT Gateway, Direct Connect, or VPN. Each VPC requiring outbound internet access must deploy its own IGW/NAT Gateway, or utilize an AWS Transit Gateway architecture.

6. Resource Cleanup & Learning Outcomes

6.1 Responsible Cloud Teardown Verification

To prevent ongoing billing charges in the ap-south-1 lab account, all provisioned resources were decommissioned following the EWB protocol in the exact reverse order of dependency:
1. Terminated all EC2 instances: ewb-bastion, ewb-app-a, ewb-app-b.
2. Disassociated and deleted VPC Peering Connection: ewb-a-b-peer.
3. Removed custom routes pointing to peering connection from all three route tables.
4. Deleted custom Security Groups: ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b.
5. Disassociated and deleted subnets: ewb-a-public, ewb-a-private, ewb-b-private.
6. Deleted custom route tables: ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private.
7. Detached and deleted Internet Gateway: ewb-igw-a.
8. Deleted both Virtual Private Clouds: ewb-vpc-a and ewb-vpc-b.

6.2 Key Architecture Takeaways

• **CIDR Hygiene:** Overlapping CIDR ranges break VPC peering at creation time; careful address space planning (/16 VPCs, /24 subnets) is critical.
• **Bastion Host Pattern:** Never copy private keys to remote instances; use SSH agent forwarding or ProxyJump for clean transit.
• **Defense in Depth:** Private workloads must have no public IPs, no default IGW routes, and security groups locked down to specific /32 source IPs.
• **Infrastructure as Code:** Both Terraform and CloudFormation templates were authored to ensure 100% reproducible deployments.

Student Declaration:

I hereby confirm that this AWS VPC Peering laboratory project was planned, deployed, tested, and analyzed by me in accordance with the EWB Cloud Labs curriculum requirements.

Submitted by: Ankit Singh Yadav **Date:** October 30, 2026 **Signature:** *Ankit S. Yadav*